

Output:

\$ gcc lab4.c

\$./a.out

Enter length: 5

Enter 5 elements:

1 2 3 4 5

Even sum: 6

Odd sum: 9

free
6/6/2023

LAB-7

6th October

1. If you wish to implement Best Fit, First Fit, Next Fit, Worst Fit memory allocation policy, it is probably best to do this by describing memory as a structure in a linked list:

struct mablk

int offset;

int size;

int allocated;

struct mablk *next;

struct mablk *prev;

} ~~type~~;

typedef struct mablk mablk;

typedef struct mablk *mabptr;

Either way, following set of prototypes give a guide as to the functionality you will need to provide:

mabptr memmchke(mabptr m, int size); // check if memory available

mabptr memalloc(mabptr m, int size); // allocate a memory block

mabptr memfree(mabptr m); // free memory block

mabptr memmerge(mabptr m); // merge two memory blocks

mabptr memsplit(mabptr m, int size); // split a memory block.

→ mabptr memmchke (mabptr m, int size) {

if (m == allocated == 0 & size > 0) {

return m;

}

mabptr memsplit (mabptr m, int size) {

if (m == size <= size) return m;

mabptr newBlock = (mabptr) malloc (sizeof (mabptr));

newBlock->offset = m->offset + size;

newBlock->size = m->size - size;

newBlock->allocated = 0;

newBlock->next = m->next;

newBlock->prev = m;

if (m->next != NULL)

m->next->prev = newBlock;

m->next = newBlock;

m->size = size;

return m;

}

mabptr memalloc (mabptr head, int size)

{ mabptr current = head, best = NULL;

while (current != NULL) {

if (!current->allocated & current->size >= size) {

if (best == NULL || current->size < best->size)

best = current;

}

current = current->next;

}

if (best != NULL) {

if (best->size > size)

best = memsplit (best, size);

best->allocated = 1;

printf ("Allocated %.1d bytes at offset %.1d in %.1d size %.1d", best->offset, best->size, best->prev, best->next);

return best;

}

```
printf("No suitable block found for i-d byte %d\n", size);
```

```
return NULL;
```

```
}
```

```
void merge(MemPtr m) {
```

```
    m->allocated = 0;
```

```
    printf("Free block at offset %d\n", m->offset);
```

```
    merge(m);
```

```
    return m;
```

```
}
```

```
void main(MemPtr m) {
```

```
    if (m->next && m->next->allocated == 0) {
```

```
        m->size += m->next->size;
```

```
        MemPtr tmp = m->next;
```

```
        m->next = tmp->next;
```

```
        if (tmp->next) {
```

```
            tmp->next->prev = m;
```

```
        free(tmp);
```

```
}
```

```
if (m->prev && m->prev->allocated == 0) {
```

```
    m = m->prev->prev;
```

```
    return m;
```

```
}
```

```
int main() {
```

```
    MemPtr head = (MemPtr) malloc(sizeof(Mem));
```

```
    head->offset = 0;
```

```
    head->size = 500;
```

```
    head->allocated = 0;
```

```
    head->next = NULL;
```

```
    head->prev = NULL;
```

```
    memAlloc(head, 100);
```

```
    memAlloc(head, 200);
```

```
    memAlloc(head, 100);
```

```
    memFree(head->next);
```

```
    memAlloc(head, 150);
```

```
    return 0;
```

```
}
```

Q19:

allocated 120 bytes at offset 0

allocated 200 bytes at offset 120

allocated 100 bytes at offset 320

allocated 100 bytes at offset 120

free block at offset 120

allocated 150 bytes at offset 120

2. Write a C-program using malloc for implementing Multi-level
feedback queue using three queues with each of them
working with different scheduling policies.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#define Q1 4
```

```
#define Q2 8
```

```
typedef struct process {
```

```
    int pid;
```

```
    int bt;
```

```
    int rt, wt, tt, queue, serv;
```

```
    struct process *next;
```

```
} process;
```

```
typedef struct queue {
```

```
    process *front;
```

```
    process *rear;
```

```
} queue;
```

```
queue createQueue() {
```

```
    queue *q = (queue*) malloc(sizeof(queue));
```

```
    q->front = q->rear = NULL;
```

```
    return q;
```

```
}
```

```
process* createProcess(int pid, int bt) {
```

```
    process *p = (process*) malloc(sizeof(process));
```

```
    p->pid = pid;
```

```

p->t = b;
p->t = b;
p->t = 0;
p->t = 0;
p->t = 0;
p->queue->level = 1;
p->next = NULL;
return p;

```

```

3
void enqueue (Queue *q, Process *p) {
    p->next = NULL;
    if (q->next == NULL)
        q->front = q->rear = p;
    else {
        q->rear->next = p;
        q->rear = p;
    }
}

```

```

3
Process* dequeue (Queue *q) {
    if (!isEmpty(q))
        return NULL;
    Process *tmp = q->front;
    q->front = q->front->next;
    if (q->front == NULL)
        q->rear = NULL;
    tmp->next = NULL;
    return tmp;
}

```

```

3
int isEmpty (Queue *q) {
    return q->front == NULL;
}
3
void print (Queue *q) {
    int time = 0;
    while (!isEmpty(q) || !isEmpty(q->next) || !isEmpty(q->next->next)) {
        Process *p = NULL;
        if (!isEmpty(q)) {
            p = dequeue(q);
            printf("in time %d: Running P/d from Q1\n", time, p->pid);

```

```

int exec_time = (p->tm > Q1)? Q1 : p->tm;
p->tm = exec_time;
time += exec_time;

```

```

if (p->tm > 0) {
    p->queue->level--;
    enqueue(Q2, p);
    printf("P/d not finished, moving to Q2\n", p->pid);
}

```

```

3
if (p->tm == time)
    printf("P/d finished at time %d\n", p->pid, time);

```

```

3
if (!isEmpty(Q2)) {
    p = dequeue(Q2);
    printf("in time %d: Running P/d from Q2\n", time, p->pid);
}

```

```

int exec_time = (p->tm > Q2)? Q2 : p->tm;
p->tm = exec_time;
time += exec_time;

```

```

if (p->tm > 0) {
    p->queue->level--;
    enqueue(Q3, p);
    printf("P/d not finished, moving to Q3\n", p->pid);
}

```

```

3
if (p->tm == time)
    printf("P/d finished at time %d\n", p->pid, time);

```

```

3
if (!isEmpty(Q3)) {
    p = dequeue(Q3);
    printf("in time %d: Running P/d from Q3\n", time, p->pid);
}

```

```

time += p->tm;
p->tm = 0;
p->tm = 0;

```

```

printf("P/d finished at time %d\n", p->pid, time);

```



```

if (offset > segtable[segment].limit) {
    printf("Error: offset %d exceeds limit of segment %d (limit = %d)\n",
           offset, segment, segtable[segment].limit);
}

```

return -1;

}

```

int physicalAddress = segtable[segment].base + offset;

```

return physicalAddress;

}

```

int main() {

```

struct segment segtable[N];

```

segtable[0].base = 1400; segtable[0].limit = 1000;

```

```

segtable[1].base = 6300; segtable[1].limit = 400;

```

```

segtable[2].base = 4300; segtable[2].limit = 400;

```

```

segtable[3].base = 3200; segtable[3].limit = 1100;

```

```

segtable[4].base = 4200; segtable[4].limit = 1000;

```

```

int seg = 2, offset = 53

```

```

int pa = 1400 + (segtable[seg].offset);

```

```

if (pa < 0)

```

```

    printf("Physical Address of %d byte of segment %d = %d\n", offset, seg, pa);

```

```

seg = 3; offset = 853;

```

```

pa = 1400 + (segtable[seg].offset);

```

```

if (pa < 0)

```

```

    printf("Physical Address of %d byte of segment %d = %d\n", offset, seg, pa);

```

```

seg = 0; offset = 1033;

```

```

pa = 1400 + (segtable[seg].offset);

```

```

if (pa < 0)

```

```

    printf("Physical Address of %d byte of segment %d = %d\n", offset, seg, pa);

```

return 0;

}

OP:

Physical Address of 53 byte of segment 2 = 4353

Physical Address of 853 byte of segment 3 = 4912

Physical Address of 1033 byte of segment 0 (limit = 1000)

4. Write a program to simulate LRU approximation page replacement using second chance algorithm. Find the total number of page faults and hit ratio for algorithm.

```

page faults

```

```

#include <stdio.h>

```

```

#include <stdlib.h>

```

```

typedef struct

```

```

    int page, ref-bit;

```

```

} frame;

```

```

int main() {

```

```

    int n, f, c, pf = 0, hit = 0;

```

```

    printf("Enter number of pages in reference string: ");

```

```

    scanf("%d", &n);

```

```

    int arr = (int *) malloc(sizeof(int) * n);

```

```

    printf("Enter the reference string: \n");

```

```

    for (int i = 0; i < n; i++) {

```

```

        scanf("%d", &arr[i]);

```

```

    }

```

```

    printf("Enter number of frames: ");

```

```

    scanf("%d", &fc);

```

```

    frame *frames = (frame *) malloc(fc * sizeof(frame));

```

```

    for (int i = 0; i < fc; i++) {

```

```

        frames[i].page = -1;

```

```

        frames[i].ref-bit = 0;

```

```

    }

```

```

    int pointer = 0;

```

```

    for (int i = 0; i < n; i++) {

```

```

        int cp = 1300;

```

```

        int found = 0;

```

```

        for (int j = 0; j < fc; j++) {

```

```

            if (frames[j].page == cp) {

```

```

                found = 1;

```

```

                hit++;

```

```

                frames[j].ref-bit++;

```

```

            }

```

```

        }

```

```
if (!found)
```

```
pf++;
```

```
while(1){
```

```
if (frames[pointer].page == -1){
```

```
frames[pointer].page = cp;
```

```
frames[pointer].ref-bit = 1;
```

```
pointer = (pointer + 1) % fe;
```

```
break;
```

```
} else if (frames[pointer].ref-bit == 0){
```

```
frames[pointer].page = cp;
```

```
frames[pointer].ref-bit = 1;
```

```
pointer = (pointer + 1) % fe;
```

```
break;
```

```
} else {
```

```
frames[pointer].ref-bit = 0;
```

```
pointer = (pointer + 1) % fe;
```

```
}
```

```
}
```

```
}
```

```
float hit-ratio = (float) hits / n;
```

```
printf("Total Page References: %d\n", n);
```

```
printf("Total Page Faults : %d\n", pf);
```

```
printf("Total Hits : %d\n", hits);
```

```
printf("Hit Ratio : %.2f\n", hit-ratio);
```

```
printf("Miss Ratio : %.2f\n", 1 - hit-ratio);
```

```
free(refr);
```

```
free(frames);
```

```
return 0;
```

```
}
```

O/P:

Enter number of pages in reference string: 9

Enter the reference string:

1 2 3 6 5 1 2 5 3

Enter number of frames: 3

Total page references : 9

Total page faults : 8

Total Hits : 1

Hit Ratio : 0.11

Miss Ratio : 0.89

12/13/10/2